EECS 3311 Software Design

Fall 2026

Sections Z

Deliverable 3 (Total Marks 100)

Overview

We assume you have already finished the development of your project. In this deliverable, we will focus on the testing of your project. The purpose of this deliverable is to practice the use of different software quality assurance, software testing, and static analysis techniques to improve the quality of your project. You are expected to finish the following tasks in this deliverable.

1. Write Junit-based test cases for the Java classes. Your test cases should achieve an average code coverage larger than 80%.
2. Run the auto-test generation tool Randoop on your project to generate test cases and further compare the auto-generated tests to your tests regarding code quality and coverage. For Randoop, please follow the attached tutorial to run it on your own course project.
3. Generate test cases for the Java classes using an AI-assisted tool. Your role is to critically analyze the AI-generated test cases for correctness, completeness, and relevance. Identify any incorrect, redundant, or missing tests and explain how these issues could affect system reliability or code coverage. Show how you incorporated, updated, or corrected the AI-generated tests to make them accurate. Explain how you integrated them into your testing framework and provide examples of your modifications with reasoning. Document the prompts you used with the AI. Describe any iterations or improvements to the prompts and explain how these changes improved the generated tests. Run your final test suite and ensure it achieves an average code coverage of at least 80% across all classes. Finally, reflect on how AI-assisted test generation helped achieve this coverage, its strengths, limitations, and lessons learned about AI-assisted software engineering.

Late Penalty

Late projects will be accepted for up to two days after the due date, with weekends counting as a single day. Submissions made up to 24 hours late will incur a 20% penalty, and submissions made up to 48 hours late will incur a 30% penalty. No submissions are accepted more than 48 hours after the deadline. The same time windows hold if there is an extension is given by the instructor.

Group Effort

This stage of the project is expected to be a group effort, with each member of the group contributing equally in a reasonable fashion. If it is determined that you are guilty of cheating on the assignment, you could receive a grade of zero with a notice of this offense submitted to the Dean of your home faculty for inclusion in your academic record.

Deliverables

Your submission, as noted above, will be in the form of an archive (e.g. zip) file. Please check e-class regularly for any changes related to the submission process. You are to complete the deliverable by:

1. Your project (i.e., source code + test cases) (40 marks)
    a. You are expected to create a test folder under your project and write at least 10 test cases for each Java class, and overall, the code coverage of your tests should be higher than 80%.
    b. Repot of the code coverage (image/screenshot) of your manual written tests.

2. The output of Randoop: (20 marks)
    a. You are expected to create another test folder, i.e., Randoop, to store all the auto-generated tests.
    b. Repot the code coverage (image/screenshot) of these automatically generated tests.

3. The testcases generated by the AI-assistant: (20 marks)
    a. You are expected to create another test folder, i.e., AI_assistant, to store all the AI-generated tests.
    b. Report on the code coverage (image/screenshot) of these AI-generated test cases.
    c. You must report which AI assistant you used, the prompts you provided, and how the AI responded to each prompt. Describe whether each generated test case was accepted, modified, or rejected, and explain your reasoning. If you went through prompt refinement steps, explain what issues you encountered with earlier prompts and how you adjusted them to arrive at a more successful setup. Where your understanding of the expected class behavior differed from what the AI generated, describe the conflict, how you evaluated it, and how you resolved it. Finally, explain how you verified the correctness of the AI-generated tests with respect to the actual behavior of your classes, and provide concrete examples of both **failed test cases** (tests that did not compile, failed at runtime, or produced incorrect assertions) and **successful test cases** (tests that passed and provided meaningful, accurate coverage).

4. Comparison between the approaches (20 marks):
    a. A report that presents a comparison between the manually written test cases, the ones generated by Randoop, and the ones generated by the AI assistant, with respect to the following dimensions:

        - **Code coverage** – Compare the coverage achieved by each approach across the non-GUI classes. Which approach covered more code, and why?

        - **Mutation score** – Compare how effectively each test suite detected injected faults. Which suit killed the most mutants?

- **Readability** – Evaluate how understandable and well-structured each test suite is. Are the tests easy to follow and maintain?
- **Usefulness** – Assess the practical value of each test suite. Do the tests catch real faults, cover meaningful scenarios, and include non-trivial assertions?

Notes of Mutation Score:

One of your D3 requirements is to compute the mutation score. You will do that in two ways:

1. For all classes, by using an automated tool like https://github.com/pitest/pitclipse
2. For several classes, by manually computing the mutation scores